

```

import random
import itertools
import collections

def load_data(filepath, k):
    edges = set()
    nodes = set()
    with open(filepath, 'r') as f:
        for line in f:
            if '{' in line:
                parts = line.strip().split()
                u, v = int(parts[0]), int(parts[1])
                edges.add((u, v))
                nodes.add(u)
                nodes.add(v)
    selected_nodes = sorted(list(nodes))[:k]
    selected_set = set(selected_nodes)
    subgraph_edges = [(u, v) for u, v in edges if u in selected_set and v in selected_set]
    return selected_nodes, subgraph_edges

def build_adj_dict(nodes, edges):
    adj = {u: [] for u in nodes}
    for u, v in edges:
        adj[u].append(v)
    return adj

filepath = "congress.edgelist"
k = 300 # Subset of nodes to extract from real-world network
nodes, edges = load_data(filepath, k)
adj_dict = build_adj_dict(nodes, edges)

def extract_subgraphs_three_nodes(nodes, adj_dict):
    found = []
    for combo in itertools.combinations(nodes, 3):
        g = [[0]*3 for _ in range(3)]
        for i in range(3):
            for j in range(3):
                if combo[j] in adj_dict.get(combo[i], []):
                    g[i][j] = 1
        if is_weakly_connected(g):
            found.append(matrix_to_tuple(g))
    return found

def adjacency_matrix_from_bits(bits, n):
    matrix = [[0]*n for _ in range(n)]
    idx = 0
    for i in range(n):
        for j in range(n):
            if i == j:
                continue
            matrix[i][j] = (bits >> idx) & 1
            idx += 1
    return matrix

def is_weakly_connected(adj):
    n = len(adj)
    visited = set()
    stack = [0]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        for neighbor in range(n):
            if adj[node][neighbor] or adj[neighbor][node]:
                stack.append(neighbor)
    return len(visited) == n

def matrix_to_tuple(adj):
    return tuple(tuple(row) for row in adj)

def canonical_form(adj):
    perms = itertools.permutations(range(len(adj)))

```

```

    canon = min(matrix_to_tuple([[adj[p[i]][p[j]] for j in range(3)] for i in range(3)]) for p in perms)
    return canon

def generate_all_unique_subgraphs(n):
    num_edges = n * (n - 1)
    total_graphs = 1 << num_edges
    seen = set()
    unique = []
    for bits in range(total_graphs):
        adj = adjacency_matrix_from_bits(bits, n)
        if any(adj[i][i] for i in range(n)):
            continue
        if not is_weakly_connected(adj):
            continue
        canon = canonical_form(adj)
        if canon not in seen:
            seen.add(canon)
            unique.append(canon)
    return unique

def print_all_unique_subgraphs(subgraphs):
    print(f"Total unique 3 node connected subgraphs (directed): {len(subgraphs)}\n")
    for i, subgraph in enumerate(subgraphs):
        print(f"Subgraph #{i+1}")
        for row in subgraph:
            print(" ", row)
        print()

real_subgraphs = extract_subgraphs_three_nodes(nodes, adj_dict)
subgraphs = generate_all_unique_subgraphs(3)
print_all_unique_subgraphs(subgraphs)

```

↩ Total unique 3 node connected subgraphs (directed) (n=3): 13

Subgraph #1
 (0, 0, 0)
 (0, 0, 0)
 (1, 1, 0)

Subgraph #2
 (0, 0, 0)
 (0, 0, 1)
 (1, 0, 0)

Subgraph #3
 (0, 0, 0)
 (0, 0, 1)
 (1, 1, 0)

Subgraph #4
 (0, 0, 0)
 (1, 0, 0)
 (1, 0, 0)

Subgraph #5
 (0, 0, 0)
 (1, 0, 0)
 (1, 1, 0)

Subgraph #6
 (0, 0, 0)
 (1, 0, 1)
 (1, 1, 0)

Subgraph #7
 (0, 0, 1)
 (0, 0, 1)
 (0, 1, 0)

Subgraph #8
 (0, 0, 1)
 (0, 0, 1)
 (1, 1, 0)

Subgraph #9
 (0, 0, 1)
 (1, 0, 0)
 (0, 1, 0)

Subgraph #10
 (0, 0, 1)

```

(1, 0, 0)
(1, 1, 0)

Subgraph #11
(0, 0, 1)
(1, 0, 1)
(1, 0, 0)

Subgraph #12

def randomize_edges(nodes, edges, num_swaps):
    edge_set = set(edges)
    edges = list(edges)
    n = len(edges)
    for _ in range(num_swaps):
        i, j = random.sample(range(n), 2)
        a, b = edges[i]
        c, d = edges[j]
        if len(set([a, b, c, d])) < 4:
            continue
        if (a, d) in edge_set or (c, b) in edge_set:
            continue
        edge_set.remove((a, b))
        edge_set.remove((c, d))
        edge_set.add((a, d))
        edge_set.add((c, b))
        edges[i] = (a, d)
        edges[j] = (c, b)
    return edges

def count_subgraphs(subgraph_list, subgraphs):
    counter = collections.Counter()
    for g in subgraph_list:
        canon = canonical_form(g)
        if canon in subgraphs:
            counter[canon] += 1
    return counter

def compute_z_scores(real_count, random_counts, subgraphs, num_randoms):
    z_scores = {}
    for subgraph in subgraphs:
        mu = sum(count.get(subgraph, 0) for count in random_counts) / num_randoms
        sigma2 = sum((count.get(subgraph, 0) - mu)**2 for count in random_counts) / num_randoms
        sigma = sigma2**0.5
        real = real_count.get(subgraph, 0)
        z = (real - mu) / sigma if sigma != 0 else 0
        z_scores[subgraph] = (real, mu, z)
    return z_scores

real_count = count_subgraphs(real_subgraphs, subgraphs)

num_randoms = 100
random_counts = []
for _ in range(num_randoms):
    rand_edges = randomize_edges(nodes, edges.copy(), len(edges))
    rand_adj = build_adj_dict(nodes, rand_edges)
    rand_subgraphs = extract_subgraphs_three_nodes(nodes, rand_adj)
    random_counts.append(count_subgraphs(rand_subgraphs, subgraphs))

def print_adjacency_matrix(matrix):
    for row in matrix:
        print(" ", row)
    print()

def classify_and_report(z_scores):
    print(f"{'Motif Index':<12} {'Real':<5} {'MeanRand':<10} {'Z-Score':<8} {'Type'}")
    print("="*60)
    for idx, (subgraph, (real, mean, z)) in enumerate(z_scores.items()):
        if z > 2:
            type = "Motif"
        elif z < -2:
            type = "Anti-Motif"
        else:
            type = "Neutral"
        print(f"#{idx:02} {real:<5} {mean:<10.2f} {z:<8.2f} {type}")

```

```

print("Adjacency Matrix:")
print_adjacency_matrix(subgraph)

z_scores = compute_z_scores(real_count, random_counts, subgraphs, num_randoms)
classify_and_report(z_scores)

```

```

(0, 0, 0)
(1, 0, 0)
(1, 0, 0)

[#04]      3322   3517.68   -3.62   Anti-Motif
Adjacency Matrix:
(0, 0, 0)
(1, 0, 0)
(1, 1, 0)

[#05]      1547   1513.31    1.33   Neutral
Adjacency Matrix:
(0, 0, 0)
(1, 0, 1)
(1, 1, 0)

[#06]     24169  24292.17   -1.16   Neutral
Adjacency Matrix:
(0, 0, 1)
(0, 0, 1)
(0, 1, 0)

[#07]      9552   8630.68    8.67    Motif
Adjacency Matrix:
(0, 0, 1)
(0, 0, 1)
(1, 1, 0)

[#08]       222   278.80   -4.13   Anti-Motif
Adjacency Matrix:
(0, 0, 1)
(1, 0, 0)
(0, 1, 0)

[#09]      1561   1644.13   -2.92   Anti-Motif
Adjacency Matrix:
(0, 0, 1)
(1, 0, 0)
(1, 1, 0)

[#10]      2325   2233.63    3.57    Motif
Adjacency Matrix:
(0, 0, 1)
(1, 0, 1)
(1, 0, 0)

[#11]      3175   2947.21    6.49    Motif
Adjacency Matrix:
(0, 0, 1)
(1, 0, 1)
(1, 1, 0)

[#12]       875   745.75    7.47    Motif
Adjacency Matrix:
(0, 1, 1)
(1, 0, 1)
(1, 1, 0)

```

Biological/structural interpretation of 3-node connected subgraphs (directed)

We classify the 13 unique 3-node connected directed graphs into **motifs**, **anti-motifs**, and **neutral structures** based on their Z-scores.

Motifs (Z-Score > 2)

These structures occur **more frequently than expected** in real networks, suggesting they are functionally important and evolutionarily selected.

Motif #07: Feedforward Loop (FFL)

- Adjacency Matrix:

- **Function:** Common in transcriptional regulatory networks and neuronal systems.
- **Insights:**
 - Filters short/erroneous signals.
 - Enables persistent activation only under sustained input.
 - Implements temporal delays and logical gates (AND/OR-like behavior).

Motif #10–#12: Dense Feedback/Control Structures

- **General Structure:** 5 or 6 edges connecting all nodes densely.
- **Insights:**
 - Facilitate mutual control or decision-making.
 - Found in tightly regulated systems (e.g., metabolic cycles, feedback loops).
 - Robust to noise and allow fine-tuned control.
 - Fully connected triads (e.g., #12) allow maximum responsiveness and memory.

Anti-Motifs (Z-Score < -2)

These subgraphs are **under-represented** in the real network compared to randomized ones, indicating that they are structurally or functionally undesirable.

Motif #00–#04: Sparse or Linear Chains

- **General Structure:** Chains like $A \rightarrow B \rightarrow C$ or disjoint regulators.
- **Insights:**
 - Lack redundancy and feedback.
 - Provide minimal control or buffering.
 - Fragile to node/edge failure.
 - Rarely found in robust biological systems.

Motif #08–#09: Cycles Without Control

- **General Structure:** Simple cycles (e.g., $A \rightarrow B \rightarrow C \rightarrow A$) without damping.
- **Insights:**
 - May induce uncontrolled oscillations or chaotic dynamics.
 - Tend to be pruned during evolution unless stabilized.
 - Rare in regulatory circuits where precise control is required.

Neutral ($-2 \leq \text{Z-Score} \leq 2$)

These appear with frequencies close to random expectations. They are **not significantly selected for or against** and may occur due to general structural tendencies.

Motif #05–#06: Moderately Complex Loops

- **Insights:**
 - Provide limited feedback or control.
 - Structurally neutral.
 - Could be precursors to more optimized motifs.

Summary of Functional Roles

Type	Role in Network	Examples
Motifs	Functional units for regulation, integration	Feedforward loops, dense triads
Anti-Motifs	Structurally unstable or inefficient	Sparse chains, pure cycles
Neutral	Not under strong structural or evolutionary pressure	Mid-level feedback chains

Biological Insight

- Motifs reflect **design principles** used by nature in regulatory, metabolic, and neural networks.
- Anti-motifs reveal **what structures are avoided**, possibly due to instability or inefficiency.
- This motif profile can serve as a **network signature** to characterize different systems.

